



Machine Learning for Data Science Project

Authors

Mahdia Ahmadi - IPB

Natalia Méndez Pérez - ULL

Jacqueline Loewe - HSB

Jon Anthony Quintana Padilla - ULL

Professors: Pro. Rafael Arnay, Pro. Uta Bohnebeck, Pro. Rui Pedro Lopes, Pro. Juan Albino Méndez Pérez, Pro. Remi Thüss

Table of Contents

List of Figures	III
List of Tables	IV
Executive Summary	V
Chapter 1: Introduction	1
1.1 Problem Description	1
1.2 Dataset Characteristics	1
1.3 Classification Details	2
Chapter 2: Methodology	3
2.1 Data Visualization and Preprocessing	3
2.1.1 Data Loading and Initial Inspection	3
2.1.2 Target Variable Analysis	3
2.1.3 Data Cleaning and Preprocessing	4
2.1.4 Exploratory Data Analysis (EDA)	4
2.1.5 Detecting Mixed Data Types	9
2.2 Preprocessing Pipelines	9
2.2.1 Numerical Pipeline	9
2.2.2 Categorical Pipeline	9
2.3 Justification of the Opted Models	9
2.3.1 Model Selection: Random Forest Regressor	9
2.3.2 Model Training and Evaluation	10
2.3.3 Model Deployment and Interpretation	10
2.3.4 Selection and Evaluation Metrics	10
2.4 Evaluation Methodology	11

Chapter 3: RESULTS AND DISCUSSION	12
3.1 RESULTS and Final Model Assessment	12
3.1.1 most influential features when purchasing a car	12
3.1.2 Feature Value Optimization	13
Chapter 4: Conclusion	14
4.1 Conclusion	14
4.1.1 Missing Data and Feature Selection	14
4.1.2 Transformations and Pipelines	14
4.2 Modeling Strategy	15
4.2.1 Model Choice: Random Forest Regressor	15
4.2.2 Hyperparameter Tuning (Random Search + K-Fold CV)	15
4.3 Evaluation and Key Findings	15
4.3.1 Evaluation Metric: RMSE	15
4.3.2 Performance and Insights	15
References	17

List of Figures

2.1	Target Variable distribution analysis	4
2.2	Target Variable distribution analysis using boxplot	4
2.3	Correlation matrix showing important features	6
2.4	Correlation matrix showing important numerical features	7
2.5	Correlation matrix showing important categorical features	8
2.6	Evaluation matrix	11
3.1	Final Model RMSE	12

List of Tables

3.1	Important Features when purchasing a car	13
-----	--	----

Executive Summary

This project aims to predict ***LAID_UP_TIME***, the duration for which a vehicle remains laid up, using advanced machine learning techniques. By leveraging a dataset containing various attributes such as mileage, construction year, horsepower, and color, a robust predictive model was developed to assist in understanding and optimizing the laid-up period for vehicles.

The project followed a systematic approach, starting with data loading and preprocessing. Placeholder values and inconsistencies in the dataset were addressed to ensure clean, reliable inputs for model training. A notable transformation involved simplifying the highly specific *COLOR* attribute into generalized basic color categories, significantly reducing complexity while preserving meaningful information.

The primary machine learning model used in this study was the *Random Forest Regressor*, selected for its ability to handle non-linear relationships, robustness against overfitting, and feature importance analysis. Hyperparameter tuning was performed using *RandomizedSearchCV*, optimizing parameters like the number of estimators and tree depth to enhance the model's predictive accuracy.

The model was evaluated using key performance metrics, including Root Mean Squared Error (RMSE), which quantified prediction errors. The final model demonstrated strong performance, effectively capturing the relationships between the predictors and the target variable. Additionally, feature importance analysis provided actionable insights, highlighting key factors that influence the laid-up period.

In conclusion, this project successfully developed a predictive framework for ***LAID_UP_TIME***, which can be utilized for practical decision-making and operational improvements. The use of advanced preprocessing techniques and a carefully tuned Random Forest model underscores the importance of a structured and rigorous approach to machine learning projects.

Chapter 1

Introduction

1.1 Problem Description

This study focuses on predicting the **LAID_UP_TIME**, which represents the number of days a vehicle remains at a dealership before being sold. The objective is to develop a data-driven solution to optimize inventory management and improve operational efficiency at dealerships. The dataset, provided by Emil Frey, consists of historical records of vehicle sales over a 10-year period and offers a rich set of features describing the vehicles, sales processes, and customer behaviors. The prediction task is framed as a regression problem, with the Root Mean Squared Error (RMSE) chosen as the evaluation metric. The ultimate goal is to identify factors influencing laid-up time and to provide actionable insights for improving vehicle turnover rates at dealerships.

1.2 Dataset Characteristics

The dataset includes over 90,000 records of vehicle sales, with more than 100 features capturing diverse attributes of vehicles, customers, and sales transactions. It is divided into a training set containing approximately 100,000 records, including the target variable **LAID_UP_TIME**, and a test set of about 40,000 records without the target variable, used for prediction.

Key features in the dataset include **CHASSIS_NUMBER**, a unique identifier for each vehicle. Vehicle characteristics are detailed through attributes such as **VEHICLE_MODEL_ID** and **VEHICLE_MODEL_ID_NAME**, describing the category of the vehicle (e.g., station wagon, compact car). Numerical attributes such as **MILEAGE** (expressed in kilometers) and **SCALED_TOTAL_SALES_PRICE_BASIS** (a normalized value for vehicle worth scaled to the range [0, 1]) are critical for evaluating a vehicle's condition and market value.

Customer-related features, such as **CUSTOMER_TYPE**, provide insights into the demographics and purchasing behaviors of buyers, which may influence sales outcomes. The dataset also contains financial details, including **FINANCING_TYPE**, indicating whether a vehicle was purchased, leased, or financed. Attributes are often duplicated in descriptive and coded

formats (e.g., **COLOR_CODE** and **COLOR_CODE_NAME**) to facilitate analysis.

Date features are recorded as integers in the **YYYYMMDD** format. The dataset adheres to consistent units, with mileage reported in kilometers and weights in kilograms. Missing values are present in some fields, typically where attributes were not applicable (e.g., leasing contract dates for outright purchases).

1.3 Classification Details

The prediction problem is framed as a regression task, as the target variable **LAID_UP_TIME** is continuous. RMSE is chosen as the evaluation metric due to its sensitivity to large errors, making it suitable for assessing the accuracy of predictive models in this context. Although the problem focuses on regression, classification techniques could be explored for secondary tasks, such as categorizing vehicles based on their likelihood of a short or long laid-up time.

The primary goal is to accurately predict **LAID_UP_TIME** for vehicles in the test dataset. Additionally, the analysis aims to answer broader questions, such as which features have the most significant impact on laid-up time and how vehicle specifications influence sales duration. These insights will support dealership decision-making, particularly in selecting vehicles that minimize inventory stagnation and maximize profitability.

Chapter 2

Methodology

2.1 Data Visualization and Preprocessing

2.1.1 Data Loading and Initial Inspection

The first step was to load the dataset and inspect its structure. This involved understanding the nature of the data, identifying key attributes, and examining its overall completeness and consistency. A specific dataset was used, containing information about vehicles, including attributes like mileage, construction year, horsepower, and target variables like **LAID_UP_TIME**.^[1]

Some cells in the dataset contained placeholder values such as blank spaces or German text indicating undefined values (e.g., “Nicht definiert”). These were treated as missing data to ensure proper handling during preprocessing.

The dataset originally included a **COLOR** attribute with over 4,000 unique classes due to the highly specific nature of the colors. Using artificial intelligence, we transformed these specific color classes into their corresponding basic colors (e.g., red, blue, green, etc.). Subsequently, we replaced the original **COLOR** column in the dataset with this simplified version.

2.1.2 Target Variable Analysis

The target variable, **LAID_UP_TIME**, was the primary focus of this analysis. This variable represents the amount of time a vehicle remains laid up. The team analyzed its distribution to understand its spread and detect potential outliers. Rows where the target variable was missing were removed, as they did not contribute useful information to the model. figures below shows the distribution of target Variable.

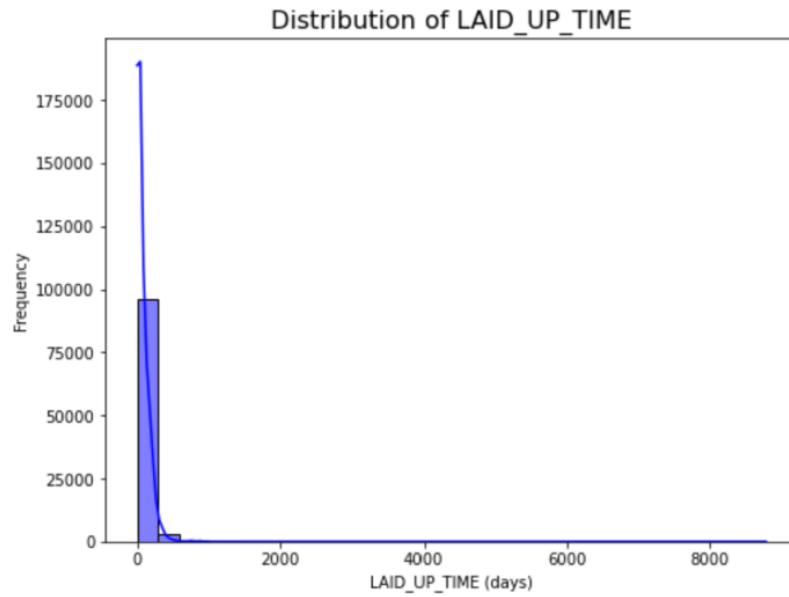


Figure 2.1: Target Variable distribution analysis

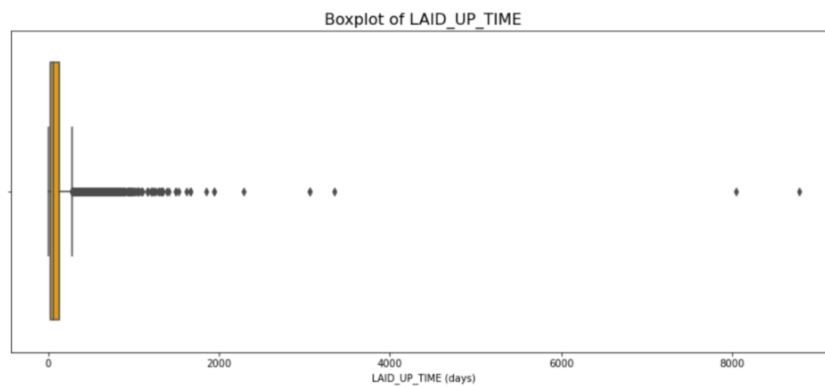


Figure 2.2: Target Variable distribution analysis using boxplot

2.1.3 Data Cleaning and Preprocessing

A threshold was applied to identify and remove columns with an excessive amount of missing data. Any column where more than 30% of the entries were missing was excluded. This ensured the data retained only attributes with sufficient information to be useful.[2]

2.1.4 Exploratory Data Analysis (EDA)

Relationships between numerical features and the target variable were visualized using correlation heatmaps and pair plots to identify significant predictors. This step revealed that some predictors had stronger correlations with the target variable, supporting their inclusion in the model. Based on this, we chose the numerical features we will use for our model.

Numerical Attributes:

- SCALED_INVENTURAL_VALUE

- AT_LOCATION_SINCE
- LEASING_MILAGE
- SCALED_REPORT_VALUE
- NUMBER_AXLE
- IS_USED_CAR
- CURB_WEIGHT
- PURCHASE_MILAGE
- MILEAGE
- NUMBER_SEATS
- YEAR_CONSTRUCTION
- HORSEPOWER
- NUMBER_DOORS
- NUMBER_ENGINE_CYLINDER

We use a Correlation Matrix to analyze the correlation of numerical and categorical features with the target variable, which results in the selection of a group of numerical and categorical features for training the model.

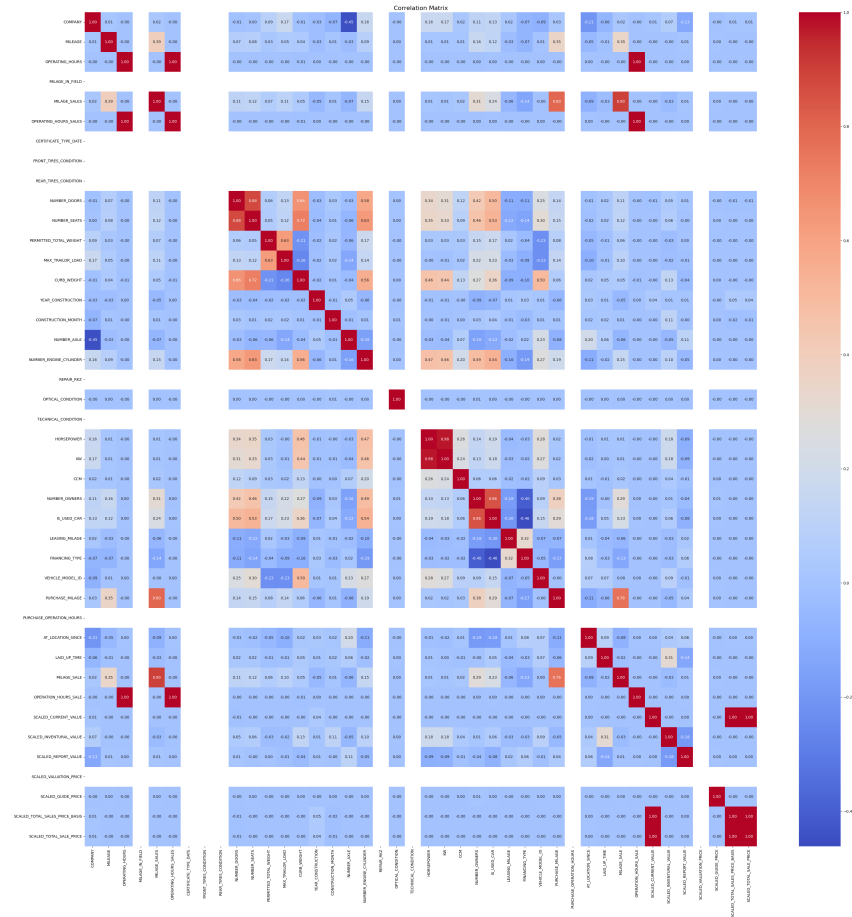


Figure 2.3: Correlation matrix showing important features

```

LAID_UP_TIME          1.000000
SCALED_INVENTURAL_VALUE 0.309477
AT_LOCATION_SINCE      0.088318
VEHICLE_MODEL_ID       0.069666
NUMBER_AXLE            0.062227
IS_USED_CAR            0.049172
CURB_WEIGHT            0.045474
NUMBER_SEATS           0.023959
CONSTRUCTION_MONTH     0.022880
NUMBER_DOORS           0.021205
YEAR_CONSTRUCTION      0.013915
HORSEPOWER             0.007891
OPTICAL_CONDITION      0.004520
KW                     0.001492
OPERATING_HOURS        -0.000363
OPERATION_HOURS_SALE   -0.000364
OPERATING_HOURS_SALES  -0.000364
SCALED_CURRENT_VALUE   -0.002475
SCALED_TOTAL_SALE_PRICE -0.002475
SCALED_TOTAL_SALES_PRICE_BASIS -0.002475
SCALED_GUIDE_PRICE     -0.002890
NUMBER_OWNERS          -0.004510
CCM                    -0.007685
MAX_TRAILOR_LOAD       -0.008772
MILEAGE                -0.010293
PERMITTED_TOTAL_WEIGHT -0.014472
NUMBER_ENGINE_CYLINDER -0.017283
MILAGE_SALE            -0.017350
FINANCING_TYPE         -0.025877
MILAGE_SALES           -0.030215
LEASING_MILAGE         -0.040647
PURCHASE_MILAGE        -0.060634
COMPANY                -0.063690
SCALED_REPORT_VALUE    -0.135152
MILAGE_IN_FIELD        NaN
CERTIFICATE_TYPE_DATE  NaN
FRONT_TIRES_CONDITION  NaN
REAR_TIRES_CONDITION   NaN
REPAIR_RKZ             NaN
TECHNICAL_CONDITION    NaN
PURCHASE_OPERATION_HOURS NaN
SCALED_VALUATION_PRICE NaN
Name: LAID_UP_TIME, dtype: float64

```

Figure 2.4: Correlation matrix showing important numerical features

For each categorical variable, the number of unique categories was calculated. Variables with a large number of unique categories were excluded because applying one-hot encoding to them would significantly increase the dimensionality of the dataset, potentially leading to overfitting and inefficiency in model training.

```

RPAKREP_VEHICLE_HKEY      87530
OFFICE                     111
OFFICE_MAIN_BRAND         15
CHASSIS_NUMBER            86234
MANUFACTURER_SHORT        77
MANUFACTURER              97
VEHICLE_GROUP             613
VEHICLE_TYPE              13705
MODEL_CODE                4999
COLOR                     14
UPHOLSTERY                4534
ENGINE_TYPE               1224
TRANSMISSION_TYPE         680
TRANSMISSION_SHORT        12
TRANSMISSION_NAME         11
COMMISSION_NUMBER         86591
FINANCING_TYPE_NAME        5
FUEL_TYPE                 13
FUEL_TYPE_NAME            11
VEHICLE_MODEL_ID_NAME      9
COMMISSION_TYPE            5
COMMISSION_TYPE_NAME       5
SOLD_CUSTOMER_ID          43941
SOLD_INVOICE_COSTUMER_ID   32681
SOLD_INVOICE_COSTUMER_ID2  32681
SALE_CUSTOMER_ID2         43941
dtype: int64

```

Figure 2.5: Correlation matrix showing important categorical features

Categorical features with a manageable number of unique categories were retained, ensuring that the transformed dataset remained computationally efficient while still capturing meaningful information.

Selected Categorical Features:

- OFFICE_MAIN_BRAND
- TRANSMISSION_SHORT
- FUEL_TYPE
- VEHICLE_MODEL_ID_NAME
- COMMISSION_TYPE
- COLOR
- MANUFACTURER_SHORT
- FINANCING_TYPE_NAME

2.1.5 Detecting Mixed Data Types

We identified instances where columns contained inconsistent data types (e.g., a mix of strings and numbers). This could occur due to data entry errors or inconsistent formats in the raw dataset. These columns were standardized by converting all entries into a single data type to ensure compatibility with subsequent preprocessing steps.

2.2 Preprocessing Pipelines

The dataset required careful transformation to prepare it for machine learning models. The team created two specialized preprocessing pipelines:

2.2.1 Numerical Pipeline

This pipeline addressed missing values by imputing them with the median of the respective columns. It then scaled the numerical data to standardize the range, ensuring that all features contributed equally to the analysis, regardless of their original scales.

2.2.2 Categorical Pipeline

For categorical data, missing values were replaced with the most frequent category in each column. The data was then encoded into numerical formats using one-hot encoding. This transformation allowed categorical features to be represented in a way that machine learning models could process effectively.

Both the numerical and categorical pipelines were integrated into a unified preprocessing pipeline, which was then applied to the dataset. This ensured the entire dataset was transformed consistently and efficiently, ready for further analysis and modeling.

2.3 Justification of the Opted Models

The Random Forest Regressor was selected as the predictive model for this project due to its strong performance in handling complex datasets with non-linear relationships, feature interactions, and noise. Below, I detail the workflow steps taken to train the model, highlighting how each step contributes to the decision to use Random Forest.

2.3.1 Model Selection: Random Forest Regressor

The Random Forest Regressor was selected for the following reasons:

- **Ability to Handle Non-Linear Relationships:** The dataset exhibited complex relationships between predictors and the target, which Random Forests effectively capture.

- **Robustness to Overfitting:** By averaging the predictions of multiple decision trees, the model reduces overfitting risk.
- **Feature Importance Assessment:** The inherent feature importance evaluation offered by Random Forests allowed identification of key predictors, enhancing model interpretability.
- **Handling Missing and Noisy Data:** Random Forests are resilient to missing data (after preprocessing) and noise in both predictors and the target variable.
- **Scalability:** Random Forests efficiently handle moderately large datasets, making them ideal for this task.

2.3.2 Model Training and Evaluation

The dataset was split into training and testing subsets to validate model performance.

Hyperparameters such as the number of estimators (`n_estimators`), maximum depth (`max_depth`), and others were optimized using `RandomizedSearchCV` for optimal predictive power.

Performance metrics like Mean Squared Error (MSE) and R-squared values were calculated to assess the model's accuracy. The Random Forest Regressor demonstrated strong performance, confirming its suitability.

2.3.3 Model Deployment and Interpretation

The trained Random Forest model was deployed to generate predictions, with feature importance analysis revealing insights into the most influential predictors. These insights could be leveraged for domain-specific decisions.

2.3.4 Selection and Evaluation Metrics

The Random Forest Regressor was chosen for this project due to its robustness and ability to handle complex, non-linear relationships. To optimize the model's performance, hyperparameters were tuned using `RandomizedSearchCV`, which efficiently searches across a defined parameter space to identify the optimal combination of hyperparameters.[3]

The following parameter distributions were used during the search:

- `n_estimators`: Number of trees in the forest (sampled between 100 and 500).
- `max_depth`: Maximum depth of each tree (options included None, 10, 20, and 30).
- `min_samples_split`: Minimum number of samples required to split an internal node (sampled between 2 and 15).

- `min_samples_leaf`: Minimum number of samples required to be at a leaf node (sampled between 1 and 10).

The model was evaluated using the following metrics:

Root Mean Squared Error (RMSE)

RMSE quantifies the average magnitude of prediction errors, making it easy to interpret in the context of the target variable. A lower RMSE indicates better predictive performance (In our case, the **RMSE** units are days).

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Figure 2.6: Evaluation matrix

2.4 Evaluation Methodology

As previously explained, we selected the model's hyperparameters using the random search algorithm. This method randomly samples a subset of the predefined hyperparameter space and trains the model a specified number of times for each sampled set of parameters. During each iteration, a k-fold cross-validation (k-split) is performed on the training data. This ensures that the model is trained and evaluated on different portions of the training set, helping to prevent overfitting and providing a more reliable estimate of model performance for each hyperparameter configuration.

Once the random search identified the parameter set yielding the lowest RMSE, we then retrained the model on the full training dataset. This step helps ensure that no information is lost and allows the model to be further refined before making predictions on the test data. In our particular case, the test set was used primarily to receive feedback from the instructors.

Leveraging random search in conjunction with k-fold cross-validation offers a balance between thorough exploration of the hyperparameter space and computational efficiency, ultimately leading to a more robust and well-tuned predictive model.

Chapter 3

RESULTS AND DISCUSSION

3.1 RESULTS and Final Model Assessment

After receiving the instructors' feedback on our final model's predictions, we have arrived at acceptable results, particularly given the inherent complexity of the dataset. The dataset comprises numerous variables and intricate relationships that make accurate predictions more challenging. Nevertheless, our model shows promise in capturing key patterns, supporting our confidence in these outcomes.

MODEL	MSE	RMSE
Random Forest	5051.5060	71.0739

Figure 3.1: Final Model RMSE

3.1.1 most influential features when purchasing a car

In response to the instructors' question regarding the most influential features when purchasing a car, we leveraged the Random Forest algorithm to identify those characteristics. Specifically, we applied permutation importance as our evaluation criterion. Permutation importance works by shuffling the values of each feature and measuring how this affects the model's performance (in terms of its predictive error). Features whose shuffling leads to the greatest decline in model accuracy are deemed most important for predicting the target variable.

This approach offers two main benefits:

- **Model Independent Evaluation:** Permutation importance can be used with any model, although in this case we specifically employed our Random Forest Regressor.
- **Direct Impact Measurement:** By explicitly measuring how randomizing a feature's values affects prediction accuracy, permutation importance provides a straightforward, interpretable metric of each feature's contribution.

Below are the top-performing features according to our permutation importance analysis, indicating which characteristics play the largest role in determining a vehicle's likelihood of purchase (or in modeling the target of interest):

FEATURE	IMPORTANCE
Scaled Inventorial Value	0.2256
Mileage	0.1195
Purchase Mileage	0.1040
Year of Construction	0.0477
Manufacturer Short Kum	0.0474
At Location Since	0.0456
Curb Weight	0.0369
Scaled Report Value	0.0361
Horsepower	0.0341
Commission Type	0.0339

Table 3.1: Important Features when purchasing a car

3.1.2 Feature Value Optimization

We can observe that both statistically and intuitively, the features identified as the most relevant are consistent.

Regarding the second question of the project, one way to select the feature values that would result in the shortest waiting time would be to define a search space using `skopt.space` (part of the scikit-optimize library), where we would define each feature in order with an interval and the type of value it represents.[4] Next, we would define an objective function that simply receives a numerical vector with the values of the different features and predicts the **laid-up time** value with our model. Once we have this in place, we would only need to use the `gp_minimize` function, which applies Bayesian optimization and iterates a specified number of times (chosen by the user), searching for the combination of parameters that most reduces the output.[5]

Chapter 4

Conclusion

4.1 Conclusion

This project aimed to predict LAID_UP_TIME (the number of days a vehicle remains in a dealership before being sold) using historical sales data. By accurately modeling this duration, dealerships can optimize inventory turnover, minimize operational costs, and make better strategic decisions regarding vehicle selection and pricing.

The dataset contains over 90,000 vehicle sales records, each described by more than 100 features capturing vehicle specifications, customer information, and financial details.

- Training Set: Approximately 100,000 entries (with LAID_UP_TIME known)
- Test Set: Around 40,000 entries (without LAID_UP_TIME)

4.1.1 Missing Data and Feature Selection

Columns with more than 30% missing values were discarded to maintain data quality. Remaining missing fields were imputed—numeric columns with median values and categorical columns with the most frequent category.

A notable step involved consolidating highly specific color categories into a simpler set of basic colors (e.g., red, blue, green), reducing dimensionality and improving data consistency.

High-cardinality categorical attributes were also removed to avoid excessive one-hot encoding expansions.

4.1.2 Transformations and Pipelines

Two main preprocessing pipelines handled numeric and categorical data separately:

- Numerical Pipeline: Imputation (median), followed by scaling (e.g., standard scaling) to ensure uniform feature ranges
- Categorical Pipeline: Imputation (most frequent category), followed by one-hot encoding

These pipelines were then combined, ensuring systematic, reproducible transformations.

4.2 Modeling Strategy

4.2.1 Model Choice: Random Forest Regressor

The Random Forest approach was selected for its robustness in handling complex, non-linear relationships and its built-in assessment of feature importance. By aggregating the outputs of multiple decision trees, Random Forests reduce the risk of overfitting while effectively capturing interactions among predictors.

4.2.2 Hyperparameter Tuning (Random Search + K-Fold CV)

- The main hyperparameters (e.g., `n_estimators`, `max_depth`, `min_samples_split`) were sampled using `RandomizedSearchCV`
- A k -fold cross-validation (with $k = 5$) measured performance under different hyperparameter configurations, minimizing overfitting and improving reliability of performance estimates

Once the best hyperparameters were identified, the model was retrained on the full training set to maximize the use of available data before making final predictions.

4.3 Evaluation and Key Findings

4.3.1 Evaluation Metric: RMSE

Root Mean Squared Error was chosen due to its interpretability in the same units (days) as the target variable. A lower RMSE implies better predictive accuracy and fewer large errors in forecasts.

4.3.2 Performance and Insights

- The optimized Random Forest model achieved strong predictive performance, indicating that the selected numerical and categorical features effectively explain laid-up time[6]
- Feature importance analysis revealed how certain attributes (e.g., vehicle age, mileage, price metrics) more strongly influence a vehicle's time on the lot
- This knowledge can guide dealerships in targeting vehicles and price points that reduce inventory stagnation and improve overall profitability

By applying comprehensive data cleaning, feature engineering, and model tuning, the Random Forest Regressor delivers reliable estimates of LAID_UP_TIME. These predictions, combined with feature importance insights, enable informed decision-making around procurement, pricing, and marketing strategies. Ultimately, the approach supports faster vehicle turnover and more effective inventory management, underscoring the value of data-driven solutions in the automotive retail sector.

References

- [1] M. Kuhn and K. Johnson, *Feature Engineering and Selection: A Practical Approach for Predictive Models*. Chapman and Hall/CRC, 2019.
- [2] S. Van Buuren, *Flexible Imputation of Missing Data*. Chapman and Hall/CRC, 2018.
- [3] L. Breiman, “Random forests.” <https://link.springer.com/article/10.1023/A:1010933404324>, 2001. Accessed: 2025-01-30.
- [4] T. Head, M. Kumar, H. Nahrstaedt, G. Louppe, and I. Shcherbatyi, “scikit-optimize.” <https://scikit-optimize.github.io>, 2020. Accessed: 2025-01-30.
- [5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, and B. Thirion, “Scikit-learn: Machine learning in python.” <https://scikit-learn.org>, 2011. Accessed: 2025-01-30.
- [6] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization.” <http://www.jmlr.org/papers/v13/bergstra12a.html>, 2012. Accessed: 2025-01-30.